Assignment II
Group I

Fabian Blasch, Nik Gabrovsek, Patrick Gyasi, Gregor Steiner

28.06.2021

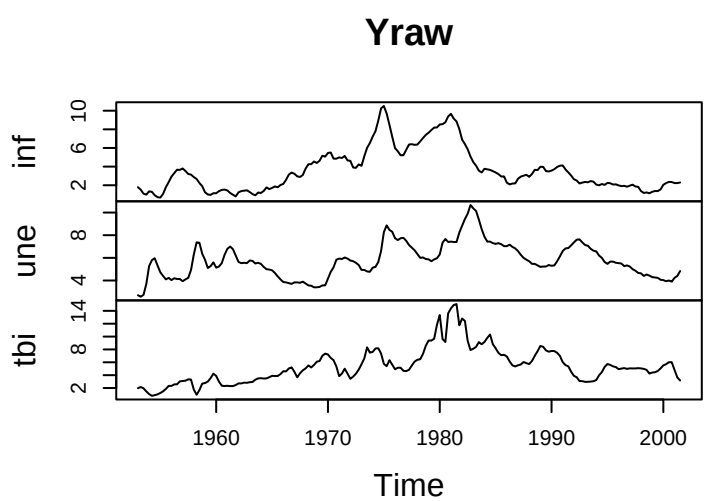
Packages

```
# Packages
get.package <- function(package){

  lapply(package, \(x){
    # check if packages are installed and if not install them
    if(!require(x, character.only = T)){
       install.packages(x)
    }
    # call package
    library(x, character.only = T)
  })

}

get.package(c("tis", "MASS", "magic", "MCMCpack", "bvarsv", "zoo"))

# since we will use Map() alot for plotting we will wrap it in invisible()
invis.Map <- function(f, ...) invisible(Map(f, ...))
```

VAR Coding

```
type <- "real" # either "sim" or "real"
###------------------------------------ Load/Simulate data ----------------------------------------------
if(type == "sim"){
  set.seed(571) #set seed to obtain the same results
  # DGP of VAR(1) process
  N <- 3 # number of endogenous variables
  persist <- 0.9 # define true coefficient of first own lag
  TT <- 100 # define total number of observations
  p <- 1

  #store object
  Yraw <- matrix(0, TT, N)
  colnames(Yraw) <- paste0("Var", 1:N)
  Yraw[1,] <- rnorm(N) #define intial observation

  A.mat <- matrix(0,N,N) # coefficient matrix
```

```
  A.mat <- matrix(rnorm(N^2, 0, 0.1), N, N)
  diag(A.mat) <- persist #first own lag

  eig.re <- max(Re(eigen(A.mat)$values))
  eig.im <- max(Im(eigen(A.mat)$values))

  Sig.low <- matrix(0,N,N) #variance-covariance matrix
  diag(Sig.low) <- 1 # true variance
  Sig.low[lower.tri(Sig.low, diag = FALSE)] <- rnorm((N*(N-1)/2), 0, 0.1) #true lower Cholesky factor
  Sig <- Sig.low%*%t(Sig.low)

  constant <- rnorm(N,0,1)

  for (tt in 2:TT){
    Yraw[tt,] <- constant + A.mat%*%Yraw[tt-1,]+Sig.low%*%rnorm(N,0,1)
  }
}else{
  if(type == "real"){
    #Real macro data of US, using the bvarsv package
    data("usmacro")
    Traw <- nrow(usmacro)
    Yraw <- usmacro
  }
}

plot.ts(Yraw)
```




```
#-----------------------------------------------------------------------------------
# useful function
mlag <- function(X,lag){
  p <- lag
  X <- as.matrix(X)
  Traw <- nrow(X)
  N <- ncol(X)
  Xlag <- matrix(0,Traw,p*N)
```

```r
  for (ii in 1:p){
    Xlag[(p+1):Traw,(N*(ii-1)+1):(N*ii)]=X[(p+1-ii):(Traw-ii),(1:N)]
  }
  return(Xlag)
}
##################################################################################
### Independent normal-Wishart prior for the VAR
##################################################################################


# specifications
plag <- 2               # number of lags
cons <- TRUE

# Create data matrices
Yraw <- as.matrix(Yraw)
Xraw <- mlag(Yraw,plag)

if(cons) Xraw <- cbind(Xraw, 1)

#first p rows are zero | conditioning on the first p observations
Y <- Yraw[(plag+1):nrow(Yraw),]
X <- Xraw[(plag+1):nrow(Xraw),]
y <- as.vector(Y)

# outside loop calculations
XX <- crossprod(X)

# get useful dimensions
M    <- ncol(Y)           # number of endogenous variables in the VAR
bigT <- nrow(Y)           # sample size, do not use T!
K    <- M*plag
k    <- ncol(X)           # number of parameters per equation
v    <- M*(M-1)/2
#---------------------------------------------------------------------------------
# Initial Values, OLS preliminaries
#---------------------------------------------------------------------------------
A_OLS <- solve(crossprod(X))%*%crossprod(X,Y)
E_OLS <- Y-X%*%A_OLS
S_OLS <- crossprod(E_OLS)/(bigT-K)

# let's have a look at OLS estimates
yfit <- X%*%A_OLS

# initialize draws with OLS estimators
A_draw <- A_OLS
S_draw <- S_OLS

# get AR variances
sigs <- numeric(length=M)
for(mm in 1:M){
  yuse <- Y[,mm,drop=FALSE]
  xuse <- cbind(X[,seq(mm,M*plag,by=M),drop=FALSE],1)
```

```r
  b    <- solve(crossprod(xuse))%*%crossprod(xuse,yuse)
  sigs[mm] <- crossprod(yuse-xuse%*%b)/(bigT-plag-1)
}
#-----------------------------------------------------------------------------------
# PRIORS
#-----------------------------------------------------------------------------------
A_prior <- matrix(0,k,M)
diag(A_prior) <- 1
a_prior <- as.vector(A_prior)



# hyperparamter for inverse Wishart
s0 <- M + 2
S0 <- (s0 - M - 1)*diag(sigs)
```

```r
# outside loop calculations
s_post    <- bigT + s0
#-----------------------------------------------------------------------------------
# MCMC setup
#-----------------------------------------------------------------------------------
nsave <- 1000               # number of saved draws
nburn <- 1000        # number of burned draws
ntot  <- nsave + nburn      # number of total draws
nhor  <- 13                 # horizon for IRFs
fhorz <- 8                  # forecasting horizon
accept <- 0


# Container for MCMC draws
A_store <- array(NA, c(nsave, k, M))
S_store <- array(NA, c(nsave, M, M))

eig_store <- numeric(length=nsave)
cou_store <- numeric(length=nsave)



# hyperprior for lambdas: Gamma(alpha, beta)
alpha.hyper <- 0.01
beta.hyper <- 0.01

# starting values for lambdas
lambda1 <- 1; lambda2 <- 1; lambda3 <- 1
lambda.draw <- c(lambda1, lambda2, lambda3)


# And storage for lambdas
lambda.store <- matrix(NA, ncol = 3, nrow = nsave)

# And parameters for the log-normal proposal
kappa <- 1
```

```r
# Predictions -- dimensions: number of draws x number of variables x forecasting horizon
yf_store <- array(NA, c(nsave,M,fhorz))

# IRFs -- dimensions: Number of draws x Number of responses x Number of structural shocks x horizon
IRFchol_store <- array(NA,c(nsave,M,M,nhor))
IRFsign_store <- array(NA,c(nsave,M,M,nhor))

set.seed(1)
for(irep in 1:ntot){



  #-----------------------------------------------------------------------------
  # Step 1: Draw S_draw | Y, A_draw from IW
  # s_overbar = T + s_underbar (s0)
  # S_overbar = (Y-XA)'(Y-XA) + S_underbar (S0)
  # SIGMA | Y ~ iW(s_overbar,S_overbar)

  S_post    <- crossprod(Y-X%*%A_draw)
  S_drawinv <- matrix(rWishart(1,s_post,solve(S_post)),M,M) # note that we can also draw from the Wisha
  S_draw    <- solve(S_drawinv)                             # and inverting leads to the inverse-Wishart
  # or using the MCMCpack
  S_draw    <- MCMCpack::riwish(s_post, S_post)


  # Step 1.1 : MH Step for lambdas

  # Draw proposal from log normal
  lambda.prop <- rlnorm(3, lambda.draw, kappa)

  # EValuate prior densities
  p.c <- dgamma(lambda.draw, alpha.hyper, beta.hyper)
  p.p <- dgamma(lambda.prop, alpha.hyper, beta.hyper)

    # Minnesota prior
  # own lags:       (lambda1/k)^2   # k == lag
  # cross lags:     (sig_i^2/sig_j^2)(lambda2/k)^2
  # deterministics: lambda3*sig_i^2

  # V_prior <- array(0,c(k,k,M))
  V_prior.c <- array(0,c(k,k,M))
  V_prior.p <- array(0,c(k,k,M))

  Map(function(V_prior, lambda){
    for(mm in 1:M){ # over all equations
      for(pp in 1:plag){ # over all lags
        for(kk in 1:M){ # over all coefficients
          # own lag
          if(mm==kk){
            V_prior[(pp-1)*M+kk,(pp-1)*M+kk,mm] <- (lambda[1]/pp)^2
          }else{
            V_prior[(pp-1)*M+kk,(pp-1)*M+kk,mm] <- (sigs[mm]/sigs[kk])*(lambda[2]/pp)^2
          }
```

```r
      }
    }
    V_prior[k,k,mm] <- lambda[3]*sigs[mm]
  }
  V_prior    <- lapply(seq(dim(V_prior)[3]), function(x) V_prior[,,x]) # array to list
  V_prior    <- Reduce(magic::adiag,V_prior)
  V_priorinv <- diag(1/diag(V_prior))

    #-----------------------------------------------------------------------------
  # Step 2: Draw A_draw | Y, S_draw from MVN
  # V_overbar = (SIGMAinv otimes X'X + Vinv_underbar)^{-1}
  # a_overbar = V_overbar (Vinv_underbar a_underbar + (SIGMAinv otimes X')y)
  V_post    <- solve(kronecker(S_drawinv, XX) + V_priorinv)
  A_post    <- V_post %*% (kronecker(S_drawinv, t(X))%*%y + V_priorinv%*%a_prior)

  A_draw     <- matrix(A_post + t(chol(V_post))%*%rnorm(k*M), k, M)

  return(list("V_overbar" = V_post,
              "a_overbar" = A_post,
              "A_draw" = A_draw))


}, list(V_prior.c, V_prior.p), list(lambda.draw, lambda.prop)) -> tmp.store

# Evaluate Likelihood
ll.c <- mvtnorm::dmvnorm(c(tmp.store[[1]][["A_draw"]]),
                         tmp.store[[1]][["a_overbar"]],
                         tmp.store[[1]][["V_overbar"]])

ll.p <- mvtnorm::dmvnorm(c(tmp.store[[2]][["A_draw"]]),
                         tmp.store[[2]][["a_overbar"]],
                         tmp.store[[2]][["V_overbar"]])




# Aceppt or reject
R.num <- prod(p.p) * ll.p *  prod(dlnorm(lambda.prop * lambda.draw, lambda.draw, kappa))
R.den <- prod(p.c) * ll.c *  prod(dlnorm(lambda.prop * lambda.draw, lambda.draw, kappa))

R <- min(1, R.num / R.den)

if(R > runif(1)){

  # accept
  lambda.draw <- lambda.prop

  # update counter
  accept <- accept + 1

}
```

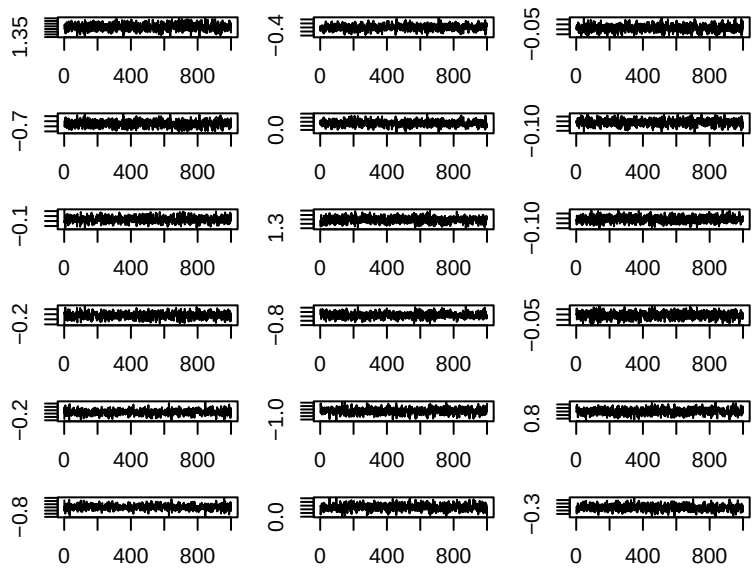
```r
# # Tune proposal variance during first 50% of burn-in
# if (irep < 0.5 * nburn) {
#   if(accept / irep > 0.3) kappa <- kappa * 1.01
#   if(accept / irep < 0.15) kappa <- kappa * 0.99
# }



#-----------------------------------------------------------------------------
# Step 3: Storage/Predictions/IRFs
if(irep > nburn){
  # Step 3a: Save draws
  A_store[irep-nburn,,] <- tmp.store[[1]][["A_draw"]]
  S_store[irep-nburn,,] <- S_draw
  lambda.store[irep-nburn,] <- lambda.draw

  #-----------------------------------------------------------------------------
  # Step 3b: Companion Matrix
  Cm <- matrix(0,M*plag,M*plag)
  Cm[1:M,] <- t(A_draw[1:(M*plag),])
  diag(Cm[(M+1):(M*plag),1:(M*(plag-1))]) <- 1
  Jm <- matrix(0,M*plag,M)
  diag(Jm) <- 1

  #-----------------------------------------------------------------------------
  # Step 3c: Do predictions and calculate fhorz-step ahead prediction density
  Mean00  <- c(Y[bigT,],X[bigT,1:(M*(plag-1))])
  Sigma00 <- matrix(0,M*plag,M*plag)
  for(ihorz in 1:fhorz){
    # first and second moments
    Mean00  <- Cm %*% Mean00
    Sigma00 <- Cm %*% Sigma00 %*% t(Cm) + Jm %*% S_draw %*% t(Jm)

    yf    <- Mean00[1:M] + t(chol(Sigma00[1:M,1:M]))%*%rnorm(M)
    yf_store[irep-nburn,,ihorz] <- yf
  }

  #-----------------------------------------------------------------------------
  # Step 3d: Check stability
  eig_store[irep-nburn] <- max(Re(eigen(Cm)$values))

  #-----------------------------------------------------------------------------
  # Step 3e: Impulse response functions
  cond.overall <- TRUE
  counter      <- 0
  MaxTries     <- 1000

  #-----------------------------------------------------------------------------
  # Step 3f: Identification via Cholesky
  shock.chol <- t(chol(S_draw))
  #Normalise shocks to 1 units
  shock.chol <- shock.chol%*%diag(1/diag(shock.chol))
```

```r
    #-----------------------------------------------------------------------------
    # Step 3g: Identification via sign-restrictions
    # draw rotation matrices Q till you find a fitting one (while-loop)
    while(cond.overall && counter < MaxTries){
      counter <- counter + 1
      # Define a rotation matrix with positive values on the main diagonal
      Rtilda <- matrix(rnorm(M^2,0,1),M,M)
      qr.object <- qr(Rtilda)
      Q <- qr.Q(qr.object)
      Q <- Q%*%diag((diag(Q)>0)-(diag(Q)<0))

      #shock is a full matrix
      shock.sign <- shock.chol%*%Q

      # Monetary Policy shock
      cond.MP <- (shock.sign[1,3]<0)*(shock.sign[2,3]>0)*(shock.sign[3,3]>0)
      # Oil price shock // aggregate supply shock
      cond.AS <- (shock.sign[1,1]>0)*(shock.sign[2,1]>0)*(shock.sign[3,1]<0)
      # Demand shock
      cond.AD <- (shock.sign[1,2]<0)*(shock.sign[2,2]>0)*(shock.sign[3,2]<0)

      #Shocks have to be mutually exclusive (orthogonal)
      cond.overall <- (cond.MP*cond.AS*cond.AD)==0
    }
    cou_store[irep-nburn] <- counter

    #-----------------------------------------------------------------------------
    # Step 3h: Compute IRFs
    # Temporary objects for state space representation
    irf.mat.chol<- irf.mat.sign  <- array(NA,c(M,M,nhor))

    # Impulse --> shock at t = 0:
    irf.mat.chol[,,1] <- shock.chol
    irf.mat.sign[,,1] <- shock.sign

    #start at t = 1, as t = 0 is the impulse shock
    Cmi <- Cm
    for(ihorz in 2:nhor){
      irf.mat.chol[,,ihorz] <- t(Jm) %*% Cmi %*% Jm %*% shock.chol
      irf.mat.sign[,,ihorz] <- t(Jm) %*% Cmi %*% Jm %*% shock.sign
      Cmi <- Cmi %*% Cm
    }
    IRFchol_store[irep-nburn,,,] <- irf.mat.chol
    IRFsign_store[irep-nburn,,,] <- irf.mat.sign
  }
  if(irep%%50==0) print(paste0("Round: ", irep,"/",ntot))
}
```

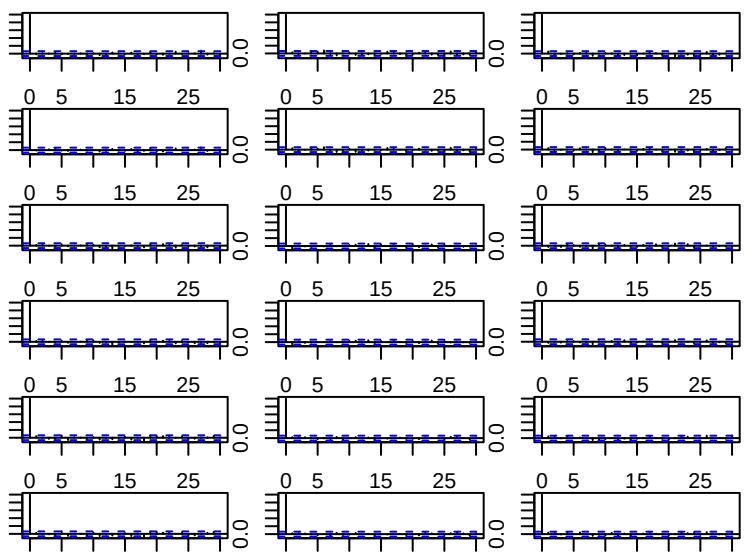
```
## [1] "Round: 50/2000"
## [1] "Round: 100/2000"
## [1] "Round: 150/2000"
## [1] "Round: 200/2000"
## [1] "Round: 250/2000"
## [1] "Round: 300/2000"
```

```
## [1] "Round: 350/2000"
## [1] "Round: 400/2000"
## [1] "Round: 450/2000"
## [1] "Round: 500/2000"
## [1] "Round: 550/2000"
## [1] "Round: 600/2000"
## [1] "Round: 650/2000"
## [1] "Round: 700/2000"
## [1] "Round: 750/2000"
## [1] "Round: 800/2000"
## [1] "Round: 850/2000"
## [1] "Round: 900/2000"
## [1] "Round: 950/2000"
## [1] "Round: 1000/2000"
## [1] "Round: 1050/2000"
## [1] "Round: 1100/2000"
## [1] "Round: 1150/2000"
## [1] "Round: 1200/2000"
## [1] "Round: 1250/2000"
## [1] "Round: 1300/2000"
## [1] "Round: 1350/2000"
## [1] "Round: 1400/2000"
## [1] "Round: 1450/2000"
## [1] "Round: 1500/2000"
## [1] "Round: 1550/2000"
## [1] "Round: 1600/2000"
## [1] "Round: 1650/2000"
## [1] "Round: 1700/2000"
## [1] "Round: 1750/2000"
## [1] "Round: 1800/2000"
## [1] "Round: 1850/2000"
## [1] "Round: 1900/2000"
## [1] "Round: 1950/2000"
## [1] "Round: 2000/2000"
```

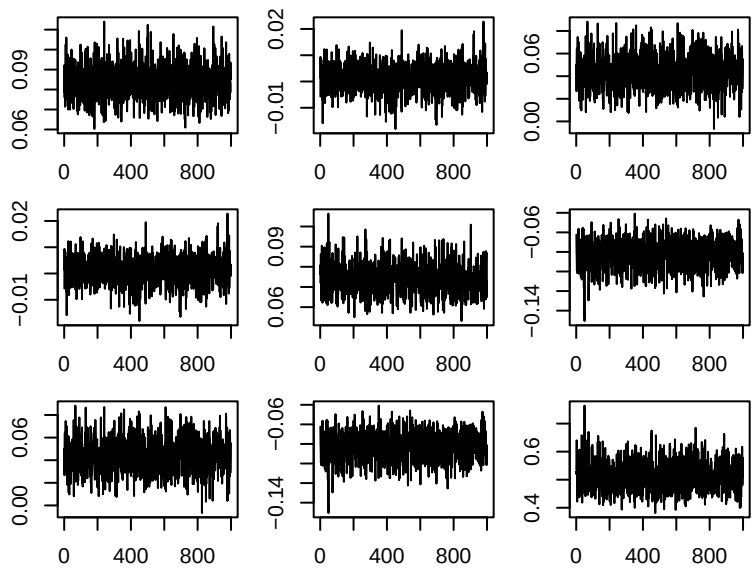
```r
# CHECK CONVERGENCE
library(coda)
crit_val <- 1.96
Z_scores <- c()

par(mfrow=c(K,3),mar=c(2,2,1,1))
for(jj in 1:M){
  for(ii in 1:K){
    plot.ts(A_store[,ii,jj])
    Z_scores[(jj-1)*K+ii] <- geweke.diag(A_store[,ii,jj])$z
  }
}
```



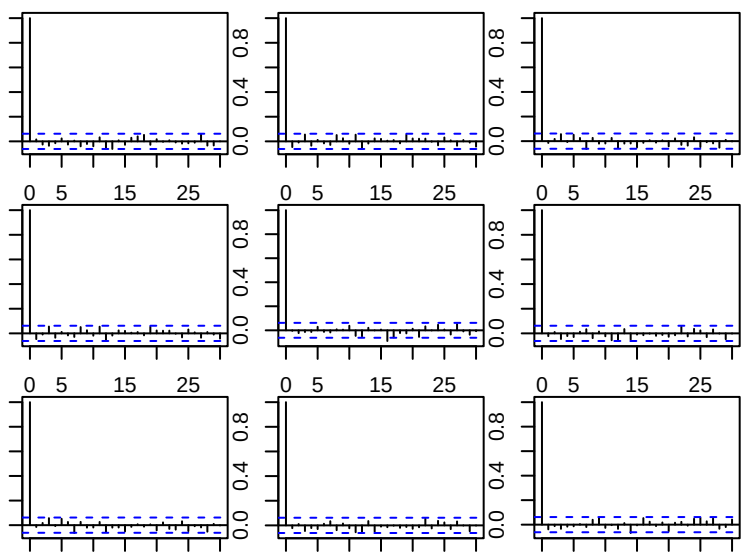

```
par(mfrow=c(K,3),mar=c(1,1,1,1))
for(jj in 1:M){
  for(ii in 1:K){
    acf(A_store[,ii,jj])
  }
}
```




```
par(mfrow=c(3,3),mar=c(2,2,1,1))
for(jj in 1:M){
  for(ii in 1:M){
    plot.ts(S_store[,ii,jj])
    Z_scores[k+(jj-1)*M+ii] <- geweke.diag(S_store[,ii,jj])$z
  }
}
```



```
par(mfrow=c(M,M),mar=c(1,1,1,1))
for(jj in 1:M){
  for(ii in 1:M){
    acf(S_store[,ii,jj])
  }
}
```

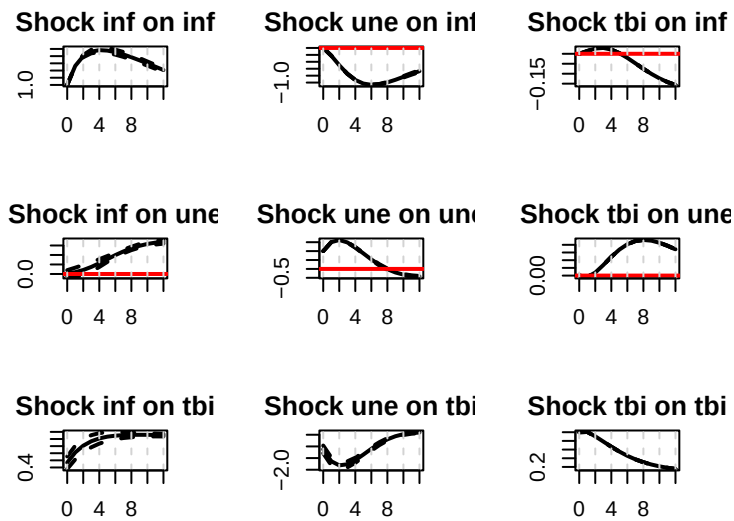


```
idx<-which(abs(Z_scores)>crit_val)
paste(length(idx), " out of ",k+M^2, " variables' z-values exceed the 1.96 threshold", " (", round(leng
```

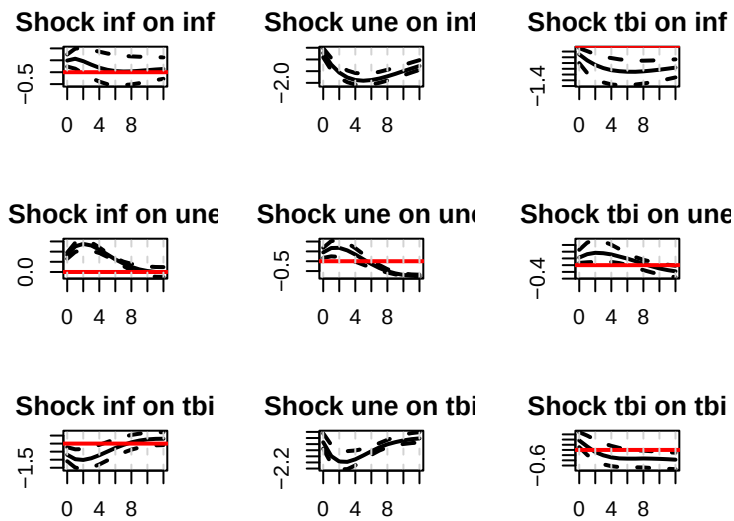
```
## [1] "1 out of 16 variables' z-values exceed the 1.96 threshold (6.25%)"
```

```
#Quantiles over the first dimension (number of saved draws)
IRFchol_low    <- apply(IRFchol_store, c(2,3,4), quantile, 0.16,na.rm=TRUE)
IRFchol_high   <- apply(IRFchol_store, c(2,3,4), quantile, 0.84,na.rm=TRUE)
IRFchol_median <- apply(IRFchol_store, c(2,3,4), median, na.rm=TRUE)
```

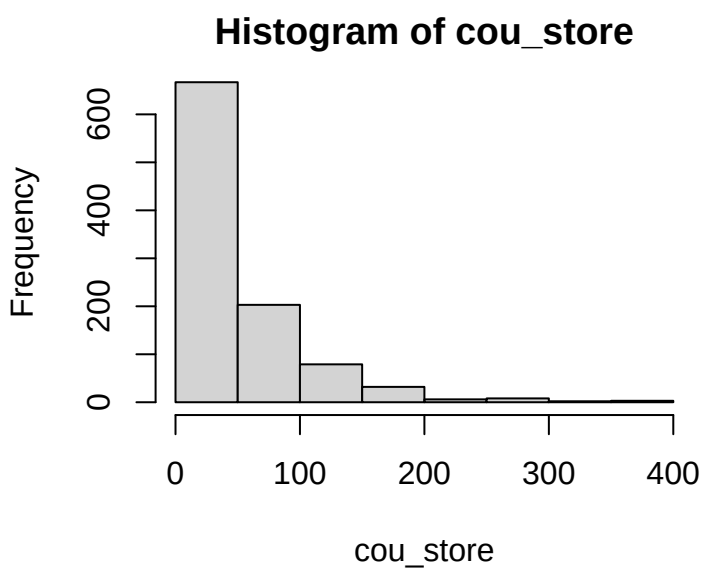
```
IRFsign_low    <- apply(IRFsign_store, c(2,3,4), quantile, 0.16,na.rm=TRUE)
IRFsign_high   <- apply(IRFsign_store, c(2,3,4), quantile, 0.84,na.rm=TRUE)
IRFsign_median <- apply(IRFsign_store, c(2,3,4), median, na.rm=TRUE)

#Start plotting the IRFs w.r.t. different shocks
par(mfrow=c(3,3),mar=c(4,4,2,2))
for(ii in 1:M){
  for(jj in 1:M){
    min1 <- min(IRFchol_low[ii,jj,])
    max1 <- max(IRFchol_high[ii,jj,])
    plot.ts(IRFchol_median[ii,jj,], ylab="", main=paste0("Shock ",colnames(Y)[jj], " on ",colnames(Y)[i
    lines(IRFchol_low[ii,jj,], lty = 2, lwd=2)
    lines(IRFchol_high[ii,jj,], lty = 2, lwd=2)
    abline(h=0,col="red",lwd=2)
    abline(v=seq(1,nhor,by=2), col="lightgrey", lty=2)
    axis(1, at=seq(1,nhor,by=2), labels=seq(0,nhor-1,by=2))
  }
}
```




```
par(mfrow=c(3,3))
for(ii in 1:M){
  for(jj in 1:M){
    min1 <- min(IRFsign_low[ii,jj,])
    max1 <- max(IRFsign_high[ii,jj,])
    plot.ts(IRFsign_median[ii,jj,], ylab="", xlab="", main=paste0("Shock ",colnames(Y)[jj], " on ",colna
    lines(IRFsign_low[ii,jj,], lty = 2, lwd=2)
    lines(IRFsign_high[ii,jj,], lty = 2, lwd=2)
    abline(h=0,col="red",lwd=2)
    abline(v=seq(1,nhor,by=2), col="lightgrey", lty=2)
    axis(1, at=seq(1,nhor,by=2), labels=seq(0,nhor-1,by=2))
  }
}
```




```
## check signs
par(mfrow=c(1,1))
hist(cou_store)
```

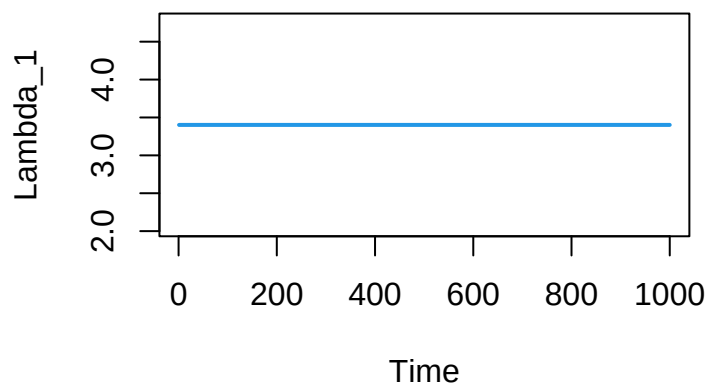


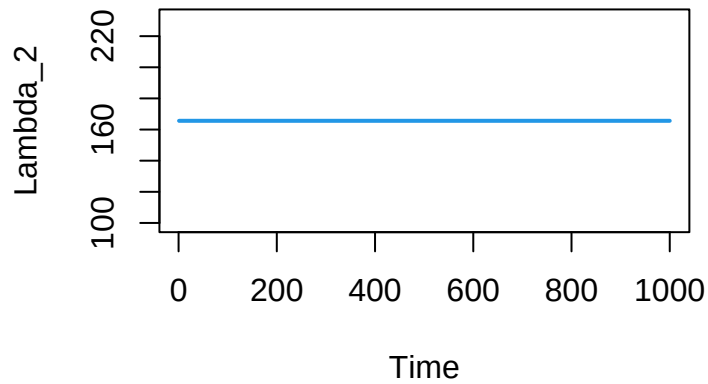

```
### plotting predictions
yf_low    <- apply(yf_store, c(2,3), quantile, 0.16, na.rm=TRUE)
yf_median <- apply(yf_store, c(2,3), quantile, 0.50, na.rm=TRUE)
yf_high   <- apply(yf_store, c(2,3), quantile, 0.84, na.rm=TRUE)

yf_low    <- cbind(t(Yraw[(bigT-5):bigT,]),yf_low)
yf_median <- cbind(t(Yraw[(bigT-5):bigT,]),yf_median)
yf_high   <- cbind(t(Yraw[(bigT-5):bigT,]),yf_high)

# xax <- c(as.character(time[(bigT-5):bigT]),paste0("t+",seq(1,fhorz)))
#
# par(mfrow=c(3,1),mar=c(5,3,2,3))
# for(ii in 1:M){
```

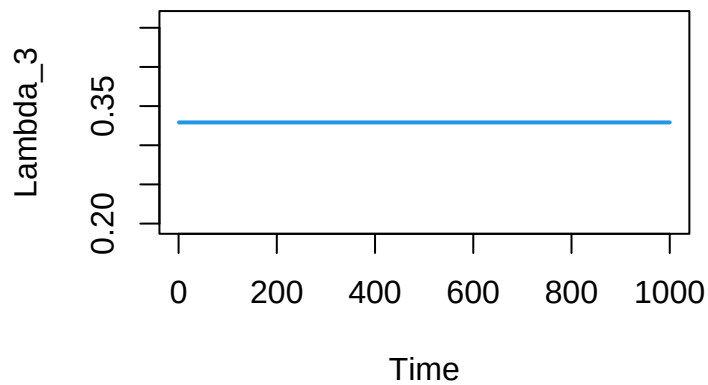
```
#   min1 <- min(yf_low[ii,])
#   max1 <- max(yf_high[ii,])
#   plot.ts(yf_median[ii,], ylim=c(min1,max1), main=colnames(Y)[[ii]], ylab="", xlab="", xaxt="n",lwd=2
#   lines(yf_low[ii,], lty=2,lwd=2)
#   lines(yf_high[ii,], lty=2,lwd=2)
#   lines(yf_median[ii,], lty=1,lwd=2)
#   axis(1, at=seq(1,14), labels=xax, las=2)
#   abline(v=seq(1,14), col="lightgrey", lty=2)
# }
```

Look at convergence of the lambdas:

```
# Traceplots
invisible(
Map(\(x, y){
  plot.ts(lambda.store[, x], ylab = y, col = 4, lwd = 2)
  }, c(1:ncol(lambda.store)), c("Lambda_1", "Lambda_2", "Lambda_3"))
)
```

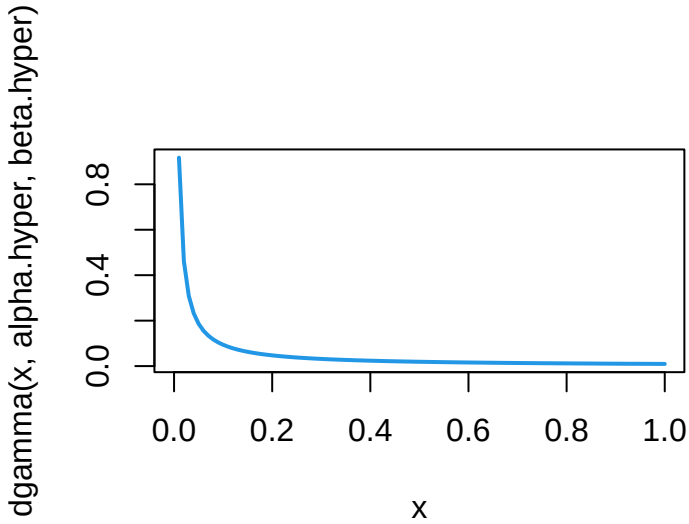






What does the prior density look like?

```
# prior density
x <- seq(0, 1, 0.01)
plot(x, dgamma(x, alpha.hyper, beta.hyper), type = "l", lwd = 2, col = 4)
```


dgamma(x, alpha.hyper, beta.hyper)
0.0
0.4
0.8
0.0
0.2
0.4
0.6
0.8
1.0
x